

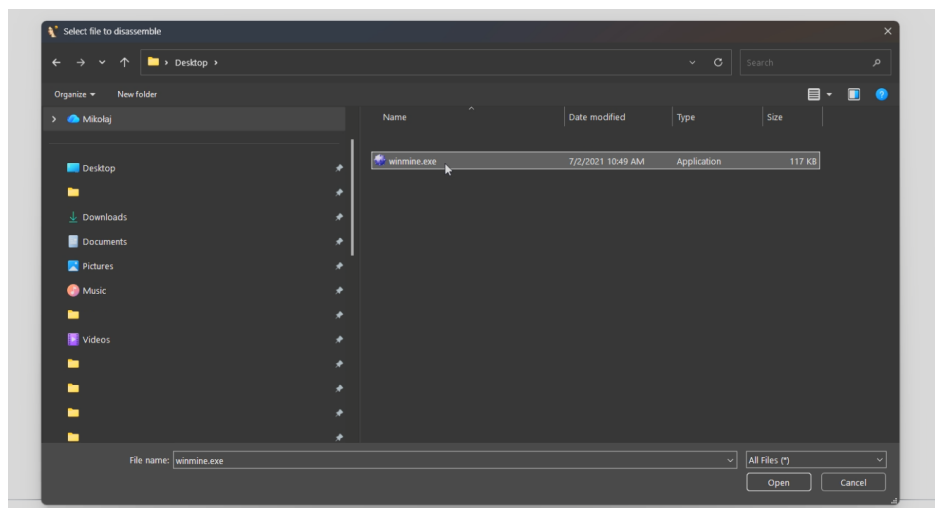
## Rozdział 3

# Analiza statyczna i dynamiczna aplikacji docelowej

Ten rozdział zawiera znakomitą większość elementu badawczego pracy. Zaczyna się on od dokumentacji procesu analizy statycznej kodu maszynowego gry Saper. Następnie, uwagę poświęcono początkowemu celowi analizy wyznaczonemu na końcu poprzedniego rozdziału. Analiza statyczna zostaje zakończona kiedy udaje się postawić hipotezę nt. lokalizacji planszy w pamięci gry i zidentyfikować funkcje prawdopodobnie odpowiedzialne za wyznaczenie indeksów komórek planszy, które zawierają miny. Następnie, w ramach analizy dynamicznej, będącej przedmiotem podrozdziału 3.2, wnioski z analizy statycznej są weryfikowane podczas uruchomienia pliku wykonywalnego gry z podłączonym debuggerem. Podrozdział 3.3 wizualizuje jak za pomocą informacji zdobytych w poprzednich podrozdziałach odtworzono z bajtów pamięci dynamicznej gry mapę planszy gry Saper z dodatkowymi informacjami, niedostępnymi dla gracza podczas zwykłej rozgrywki. Podrozdział 3.4 skupia się na dekompilacji funkcji odpowiedzialnych za oznaczanie pól na mapie przez gracza w celu możliwości zaimplementowania funkcjonalności automatycznego oznaczenia min w aplikacji w rozdziale 4. Rozdział 3 zakończony jest podsumowaniem osiągnięć ze wszystkich analiz.

## 3.1. Analiza statyczna

Analizę statyczną rozpoczęto od załadowania pliku wykonywalnego gry Saper do dezasemblera IDA Free, jak widoczne na zrzucie ekranu 4-1.



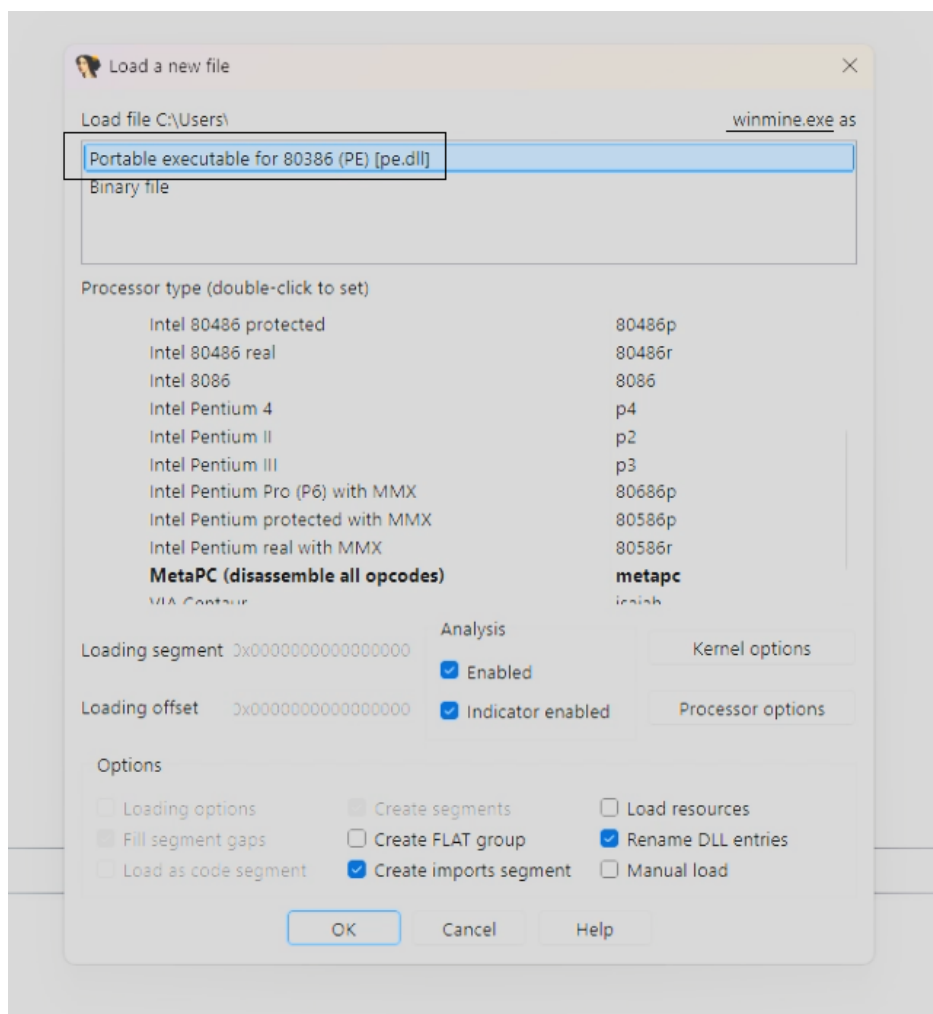
Rysunek 3.1: Wybór pliku „winmine.exe” do dezasemblacji przez IDA

Format pliku został poprawnie rozpoznany przez IDA Free jako „Portable Executable for 80386” (jak widoczne na zrzucie ekranu 4-2), co w technicznym żargonie oznacza standardową 32-bitową aplikację dla systemów Windows. Wybrano opcje domyślne.

Koncentrując się na wyznaczonym celu początkowym analizy, naszym kolejnym krokiem będzie znalezienie miejsca w pamięci gdzie przechowywane są informacje o polu minowym. Aby osiągnąć ten cel, warto przeprowadzić eksperyment myślowy. Należy rozważyć, w jaki sposób zaimplementowalibyśmy pole jako programista gry Saper? Odtworzenie gry, dla przykładowej planszy o wymiarach  $9 \times 9$ , wymaga od programisty przechowywania dla każdej komórki właściwości, takich jak:

- współrzędne (indeks wiersza i kolumny)
- stan interakcji (pole nieodkryte, odkryte, oznaczona flaga)
- zawartość logiczna (pole puste, cyfra, mina)

W celu uproszczenia architektury, dwa ostatnie parametry można połączyć w jeden atrybut reprezentujący warianty stanów i zawartości komórek (np. „nieodkryta-mina” lub „odkryta-cyfra”). Dla tak sprecyzowanych wymagań zarysowuje się naturalne rozwiązanie programistyczne – tablica dwuwymiarowa liczb całkowitych. Wybierając taką strukturę, indeksy tablicy odpowiadałyby bezpośrednio współrzędnym komórki, a przechowywana w niej wartość

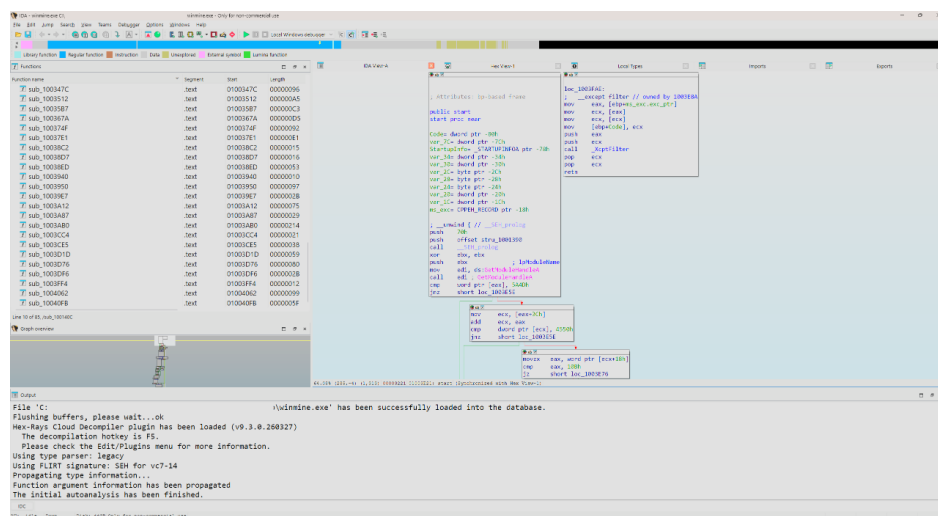


Rysunek 3.2: okno ładowania pliku PE przez IDA, ustawienia domyślne

liczbowa definiowałaby stan pola. Taki typ danych w pełni wystarcza do poprawnego funkcjonowania logiki gry Saper.

Sformułowanie powyższej hipotezy nie rozwiązuje jednak problemu jakim jest zlokalizowanie w pliku binarnym fragmentów kodu odpowiedzialnych za odczyt i modyfikację struktury planszy gry. Z pewnością, możemy potwierdzić istnienie takich fragmentów kodu – najprawdopodobniej funkcji uzyskujących dostęp do „pola minowego” – muszą być one wykonywane zarówno przed rozpoczęciem każdej kolejnej rozgrywki (w celu wyczyszczenia planszy i wygenerowania nowych pozycji min), jak i w jej trakcie, kiedy gracz swoimi ruchami zmienia stan poszczególnych komórek.

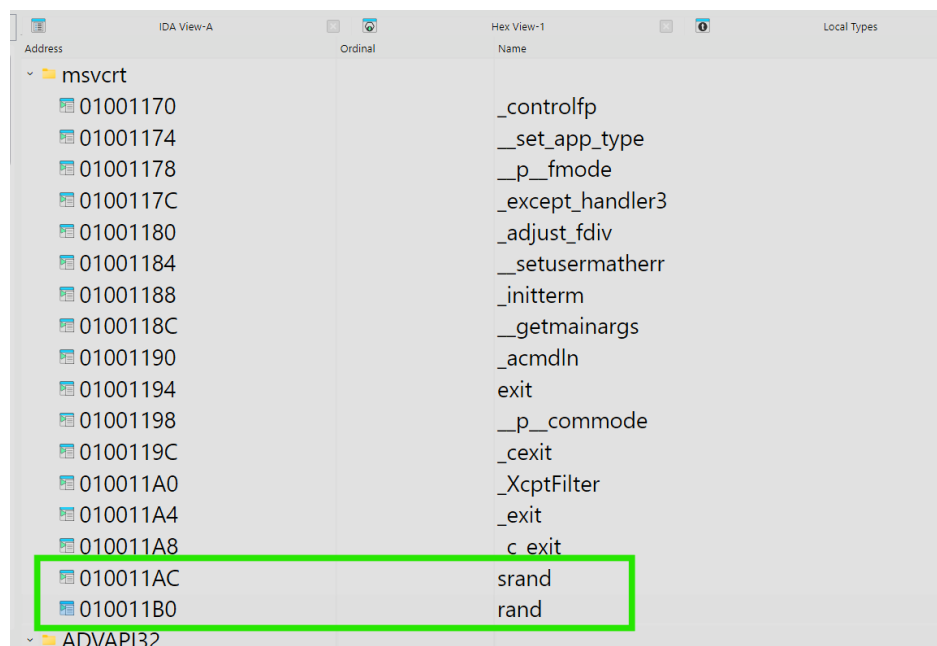
Kontynuując analizę z perspektywy inżynierii oprogramowania, należy zauważyć, że proces rozmieszczania min na planszy bezdyskusyjnie opiera się na losowaniu indeksów pól. Ponieważ mechanika gry nie wymaga kryptograficznie bezpiecznej losowości, twórcy Saperu najpewniej wykorzystali standardowy generator liczb pseudolosowych (ang. pseudo-random



Rysunek 3.3: IDA Free, widok początkowy po załadowaniu pliku „winmine.exe”

number generator), wykorzystując do tego celu bibliotekę środowiska programistycznego (np. bibliotekę stdlib języka C).

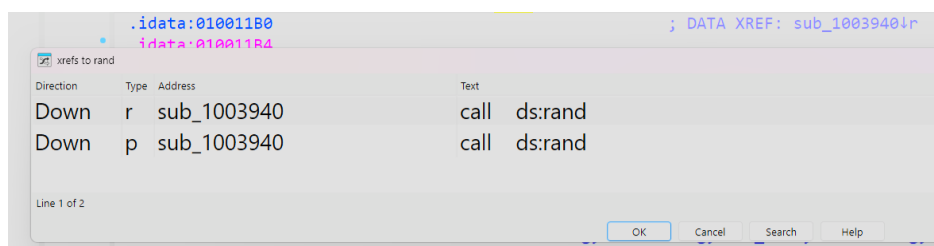
Zgodnie z teoretycznymi fundamentami przedstawionymi w rozdziale drugim, informacje o funkcjach zewnętrznych importowanych przez plik wykonywalny podczas jego uruchamiania są mapowane w strukturze tabeli adresów importu (ang. Import Address Table – IAT) formatu PE. W środowisku IDA Free, informacje te dostępne są w zakładce View > Open subviews > Imports.



Rysunek 3.4: tabela adresów importu (ang. IAT) pliku gry Saper

Analiza tabeli IAT badanego pliku rzeczywiście wykazuje import dwóch powszechnie znanych funkcji służących do generowania liczb pseudolosowych: `srand` oraz `rand`. Pochodzą one z biblioteki `msvcrt.dll` (Microsoft C Runtime Library), która dostarcza implementację standardowej biblioteki języka C w środowisku Windows. Zgodnie z dokumentacją, funkcja `srand` wykorzystywana jest do inicjalizacji ziarna (ang. seed) generatora, natomiast funkcja `rand` zwraca właściwe, wyliczane wartości pseudolosowe.

Należy jednak zauważyć, że sam fakt importowania tych funkcji nie stanowi ostatecznego dowodu na ich wykorzystanie w algorytmie rozmieszczania min. W celu weryfikacji tej hipotezy, niezbędne jest prześledzenie przepływu sterowania i zidentyfikowanie wszystkich miejsc w kodzie, z których aplikacja odwołuje się do symbolu `rand`. W środowisku IDA Free zrealizowano to poprzez nawigację do adresu importu `0x010011B0` w widoku Imports i wykorzystanie wbudowanego mechanizmu wyszukiwania referencji krzyżowych (ang. Cross-References lub XREFs). Funkcjonalność ta pozwala na wygenerowanie listy wszystkich adresów w statycznej przestrzeni pamięci pliku, które zawierają instrukcję wywołania lub jakiegokolwiek inne odwołanie do interesującego nas symbolu.

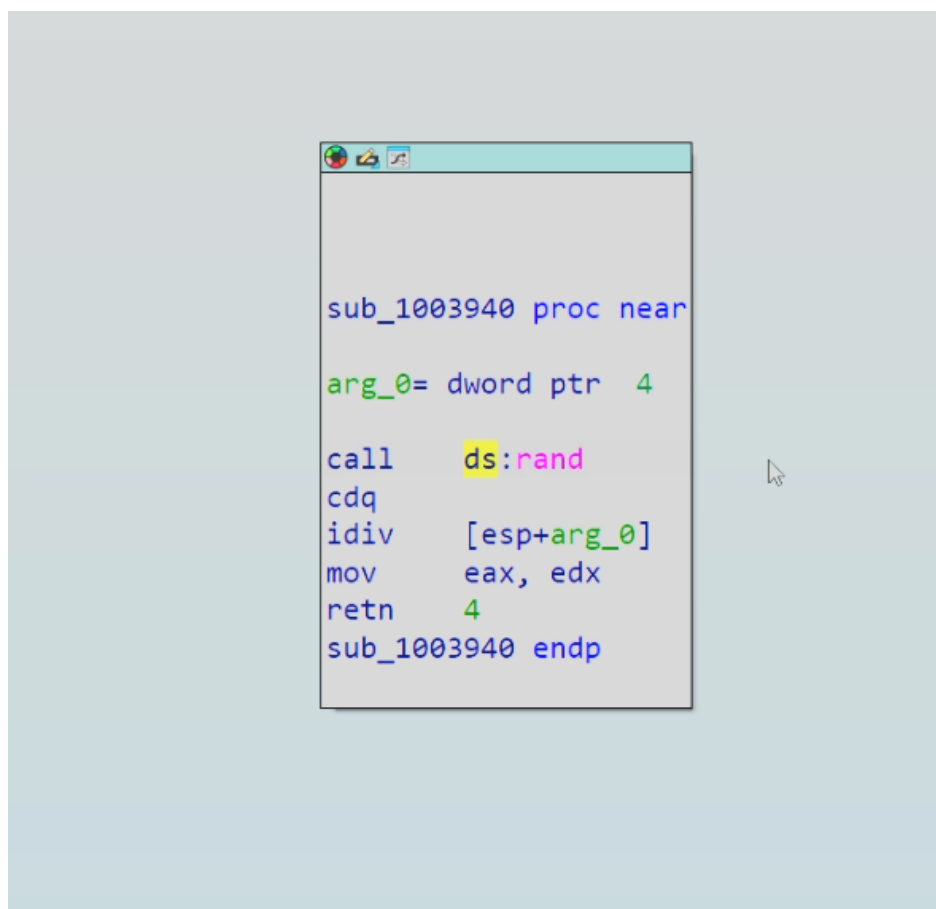


Rysunek 3.5: lista referencji krzyżowych (XREFs) do symbolu `rand`

Zgodnie z wygenerowaną listą referencji (rys. 3.5), w całej przestrzeni pliku wykonawczego `winmine.exe` są zaledwie dwa odwołania do symbolu `rand`. Oba przypadki to bezpośrednie wywołania tej funkcji, zlokalizowane wewnątrz jednej funkcji, której środowisko IDA – z powodu utraty oryginalnych symboli podczas kompilacji – automatycznie przypisało nazwę `sub_1003940`. Fakt ten stanowi znaczące ułatwienie procesu analitycznego, ponieważ wskazuje, że cały kod maszynowy powiązany z generowaniem liczb pseudolosowych jest przechowywany w obrębie jednego bloku. Taka struktura minimalizuje również narzut czasowy w przypadku konieczności ewentualnego odrzucenia badanej hipotezy. W związku z tym poddano szczegółowej inspekcji wspomnianą funkcję `sub_1003940`. Rysunek 3.6 prezentuje widok jej dezasemblacji.

Korzystając z dołączonego do programu IDA Free dekompileatora, możemy uzyskać pseudokod bliski językowi C. Jest on łatwiejszy w analizie.

Podsumowując, funkcja `sub_1003940` przyjmuje jeden parametr typu `int`, tymczasowo



Rysunek 3.6: widok dezasemblacji procedury sub\_1003940

nazwany a1. Zwraca ona liczbę całkowitą typu int. Działanie funkcji można opisać następująco: sub\_1003940 wywołuje funkcję rand() i uzyskuje wygenerowaną liczbę pseudolosową, następnie zwraca wynik reszty z dzielenia (operacji modulo) tej liczby przez argument a1. Upraszczając: sub\_1003940 zwraca liczbę pseudolosową większą lub równą zero i mniejszą niż podany argument a1. Udokumentujemy więc nasze znaleziska za pomocą opcji „Rename global item...” dostępnej pod klawiszem „N” oraz „Edit func comment” dostępnej pod klawiszem „/”. Zgodnie z powyższą analizą nadamy naszej funkcji nazwę rand\_0 i opatrzymy krótkim komentarzem w języku angielskim (jak widoczne na rys 3.8).

Użycie powyższych opcji do dokumentacji znalezisk, propaguje zmiany na cały plik projektowy IDA. Dzięki temu, od tej pory, wszystkie wystąpienia powyższej funkcji będą korzystać z nazwy rand\_0 zamiast sub\_1003940 oraz wyświetlać pierwszą linię dodanego przez nas komentarza. To pomaga nam zwiększyć czytelność i zrozumienie kodu maszynowego oraz zmniejsza przytłoczenie informacjami. Funkcja rand\_0 jest więc swego rodzaju wrapperem do funkcji rand() z stdlib. Tak jak, w poprzednich krokach użyliśmy opcji „List all cross references to...”, żeby znaleźć wywołania funkcji rand() i w efekcie udokumentować

```

1 int __stdcall sub_1003940(int a1)
2 {
3     return rand() % a1;
4 }
int a1; // [esp+4h] [ebp+4h] ISARG

```

Rysunek 3.7: pseudokod C procedury sub\_1003940, uzyskany przez dekompilekator IDA

```

1 // generate Pseudo Random number
2 // returns:
3 // - int bigger or equal to 0 and smaller than a1
4 int __stdcall rand_0(int a1)
5 {
6     return rand() % a1;
7 }

```

Rysunek 3.8: udokumentowana przez nas funkcja rand0, w pliku projektowym IDA

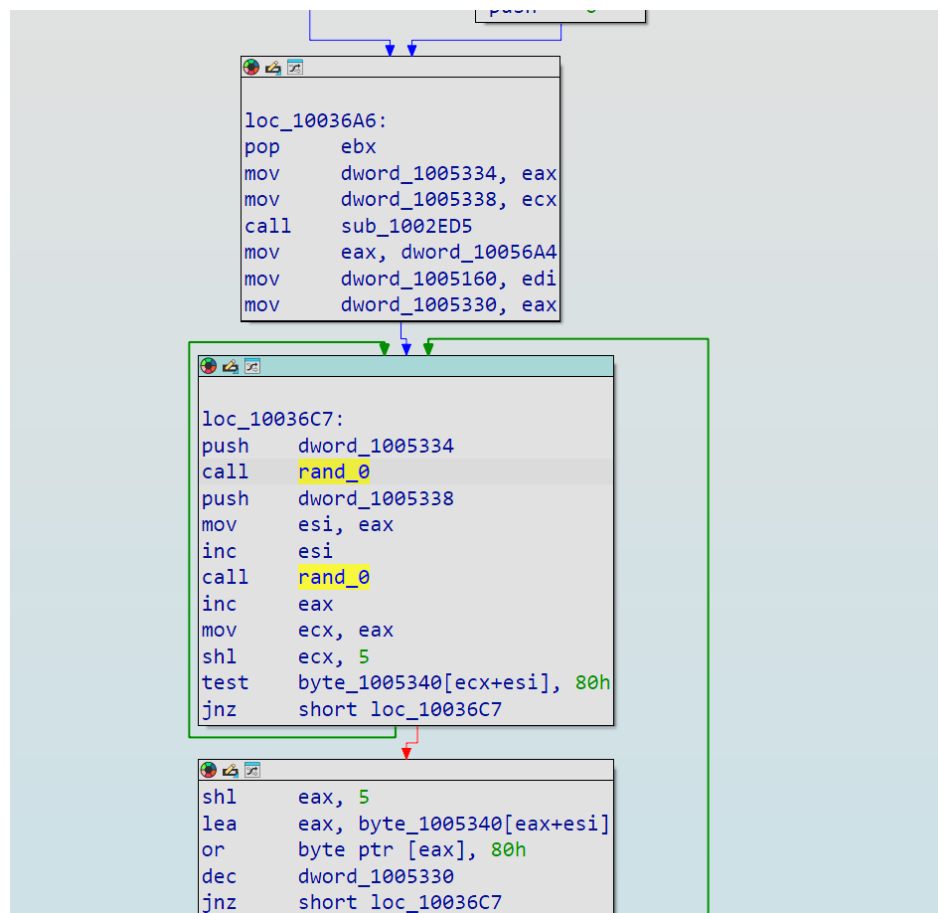
funkcję rand\_0(int a1), tak samo będziemy chcieli postąpić teraz biorąc tym razem pod lupę wszystkie wywołania funkcji rand\_0(int a1).

| Direction | Type | Address        | Text                                          |
|-----------|------|----------------|-----------------------------------------------|
| Up        | p    | sub_100367A+53 | call rand_0; // generate Pseudo Random number |
| Up        | p    | sub_100367A+61 | call rand_0; // generate Pseudo Random number |

Rysunek 3.9: lista referencji krzyżowych (XREFs) do symbolu rand0

Warto zauważyć, wartość jaką wnosi dokumentowanie analizowanego kodu maszynowego. W okienku xrefs to rand\_0 (rys 3.9) widzimy wszystkie odniesienia w winmine.exe do funkcji rand\_0. Szczęśliwie dla nas, podobnie jak przedtem te odniesienia to jedynie dwa wywołania rand\_0, na dodatek znowu w jednej funkcji, tym razem funkcji o nazwie sub\_100367A.

Postawmy więc kolejną hipotezę, że zmienne lokalne v1 i v2 przechowują indeksy rzędów i kolumn komórek planszy, w których znajdują się miny. Jeśli ją potwierdzimy to będzie oznaczało, że dword\_1005334 oraz dword\_1005338 wyznaczają liczbę rzędów oraz kolumn. Aby uzyskać dalszy postęp, przetestujemy naszą hipotezę przy wykorzystaniu analizy dynamicznej.



Rysunek 3.10: widok dezasemblacji procedury sub\_100367A



```

8  dword_1005164 = 0;
9  if ( dword_10056AC == dword_1005334 && uValue == dword_1005338 )
10     v4 = 4;
11  else
12     v4 = 6;
13  v0 = v4;
14  dword_1005334 = dword_10056AC;
15  dword_1005338 = uValue;
16  sub_1002ED5();
17  dword_1005160 = 0;
18  dword_1005330 = dword_10056A4;
19  do
20  {
21      do
22      {
23          v1 = rand_0(dword_1005334) + 1;
24          v2 = rand_0(dword_1005338) + 1;
25      }
26      while ( byte_1005340[32 * v2 + v1] < 0 );
27      byte_1005340[32 * v2 + v1] |= 0x80u;
28      --dword_1005330;
29  }
30  while ( dword_1005330 );
31  dword_100579C = 0;
32  dword_1005330 = dword_10056A4;
33  dword_1005194 = dword_10056A4;
34  dword_10057A4 = 0;
35  dword_10057A0 = dword_1005334 * dword_1005338 - dword_10056A4;
36  dword_1005000 = 1;
37  sub_100346A(0);
38  return sub_1001950(v0);
39 }

```

Rysunek 3.11: pseudokod C procedury sub\_100367A, uzyskany przez dekompilator IDA

## 3.2. Analiza dynamiczna

Analiza dynamiczna w inżynierii wstecznej oprogramowania różni się od statycznej tym, że analizowane oprogramowanie jest uruchamiane na komputerze badacza i analizowane podczas jego wykonywania. Ten rodzaj analizy okazuje się niezbędny przy programach spakowanych, zaciemnionych czy używających innych technik mających na celu ukrycie prawdziwego działania.

Ten segment naszej analizy rozpoczniemy od przyglądnięcia się zmiennym `v1` oraz `v2`, konkretniej wartościom jakie są im przypisywane podczas wywołania udokumentowanej przez nas funkcji `rand_0(int a1)` podczas wykonania programu. Przyjrzymy się również wartościom zmiennych `dword_1005334/dword_1005338`, które wyznaczają górną granicę zakresu losowanych liczb.

Naszym celem będzie zatrzymanie wykonywania programu na interesującej nas instrukcji i podglądnięcie wartości jakie są przypisywane do interesujących nas zmiennych: `v1`, `v2`, `dword_1005334`, `dword_1005338`

Zacznijmy od zmiennych `dword_1005334`, `dword_1005338`.

Jak widać w 23 i 24 linijce pseudokodu C wygenerowanego z kodu maszynowego przez IDA, są one podawane jako argument do funkcji `rand_0`. Zmiennej `v1` w 23 linijce przypisywana jest wartość wyrażenia `rand_0(dword_1005334) + 1`. Wyrażenie `rand_0(x)` zwróci liczbę pseudolosową z zakresu  $\langle 0, x \rangle$ , ponieważ w wyrażeniu na końcu mamy `+ 1`, całe wyrażenie zwróci liczbę z zakresu  $\langle 1, x + 1 \rangle$ . Z powodu iż wiemy, że będzie to liczba całkowita zakres wartości jakie zwróci wyrażenie możemy uprościć do  $\langle 1, x \rangle$ . Przekładając to na nasze konkretne wywołania oznacza to, że zmienna `v1` będzie przechowywać liczbę całkowitą z zakresu  $\langle 1, \text{dword\_1005334} \rangle$ , a zmienna `v2`, analogicznie przyjmie wartość z zakresu  $\langle 1, \text{dword\_1005338} \rangle$ .

Interesujące nas zmienne z przedrostkiem „`dword`” występują jeszcze w kilku innych niż linijki 23-24 miejscach wewnątrz funkcji `sub_100367A`. W linijce 9 są one porównywane a w linijkach 14-15 są im przypisywane właściwe wartości, których program używa do ograniczenia zakresu zmiennych `v1`, `v2` jak opisano wyżej.

Przejdziemy więc z powrotem na widok dezasemblacji zamiast pseudokodu C, żeby zatrzymać wykonywanie programu na konkretnej instrukcji kodu maszynowego. Interesują nas linijki 14-15 pseudokodu czyli przypisanie wartości zmiennym `dword`,

Klikamy prawym przyciskiem myszy na interesującą nas linijkę 15 i wybieramy opcję `Synchronize With > IDA View-A, Hex-View 1` dzięki czemu po przejściu na widok dezasemblacji interesująca nas instrukcja zostanie podświetlona na zielono.

Następnie, aby zatrzymać wykonywanie programu na instrukcjach `mov dword..., eax/ecx`

```

1 int sub_100367A()
2 {
3     int v0; // ebx
4     int v1; // esi
5     int v2; // eax
6     int v4; // [esp-4h] [ebp-10h]
7
8     dword_1005164 = 0;
9     if ( dword_10056AC == dword_1005334 && uValue == dword_1005338 )
10         v4 = 4;
11     else
12         v4 = 6;
13     v0 = v4;
14     dword_1005334 = dword_10056AC;
15     dword_1005338 = uValue;
16     sub_1002ED5();
17     dword_1005160 = 0;
18     dword_1005330 = dword_10056A4;
19     do
20     {
21         do
22         {
23             v1 = rand_0(dword_1005334) + 1;
24             v2 = rand_0(dword_1005338) + 1;
25         }
26         while ( byte_1005340[32 * v2 + v1] < 0 );
27         byte_1005340[32 * v2 + v1] |= 0x80u;
28         --dword_1005330;
29     }
30     while ( dword_1005330 );
31     dword_100579C = 0;
32     dword_1005330 = dword_10056A4;
33     dword_1005194 = dword_10056A4;
34     dword_10057A4 = 0;
35     dword_10057A0 = dword_1005334 * dword_1005338 - dword_10056A4;

```

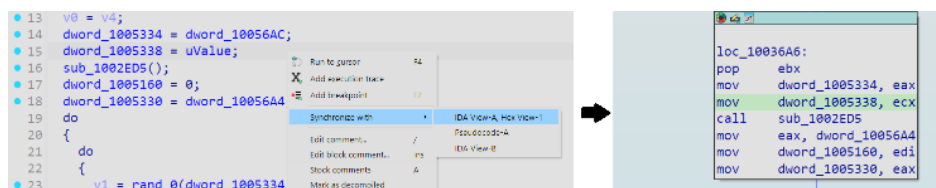
00002AA7:sub\_100367A:14 (10036A7)

Rysunek 3.12: pseudokod C procedury sub\_100367A, z zaznaczonymi zmiennymi dword\_...

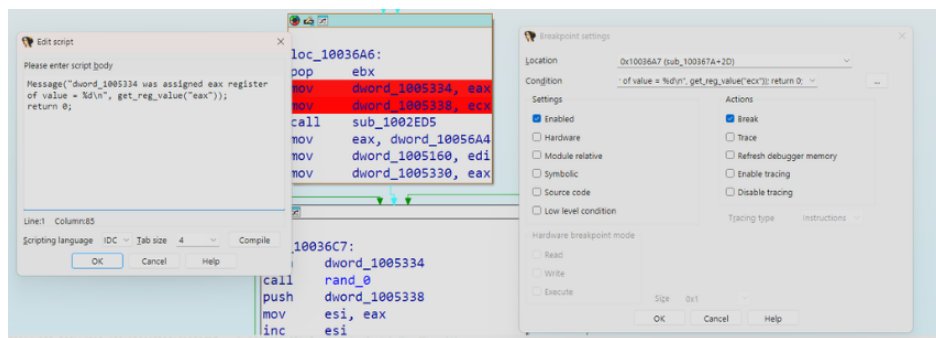
klikamy prawym przyciskiem myszy na każdą z linijek i wybieramy opcję Add Breakpoint. Dla zautomatyzowania procesu dodamy za pomocą opcji Edit Breakpoint skrypt w języku IDC, który podczas wykonywania interesującej nas instrukcji wyświetli w oknie Output krótką informację o wartości rejestrów procesora odpowiednio eax dla zmiennej 1005334 oraz ecx dla zmiennej 1005338.

Następnie uruchomimy program za pomocą wbudowanego w IDA debugger za pomocą zielonej strzałki na pasku narzędzi na samej górze ekranu. Gra uruchamia się a my widzimy, że skrypt który napisaliśmy pokazuje, że do obu zmiennych została przypisana wartość 9. Zwróćmy uwagę, że  $9 \times 9$  to dokładnie wymiary planszy gry Saper na poziomie „Beginner”. Bez wyłączania programu zmienimy poziom w grze na „Intermediate”. W tym trybie plansza ma 16 rzędów i 16 kolumn. I dokładnie takie wartości pokazuje nam nasz skrypt wykonywany przy breakpointie.

Zatem zmienne dword, przechowują wysokość i szerokość planszy czyli odpowiednio ilość rzędów i ilość kolumn. Nadal pozostaje pytanie, która zmienna odpowiada za wysokość a która za szerokość?



Rysunek 3.13: Synchronizacja widoków IDA

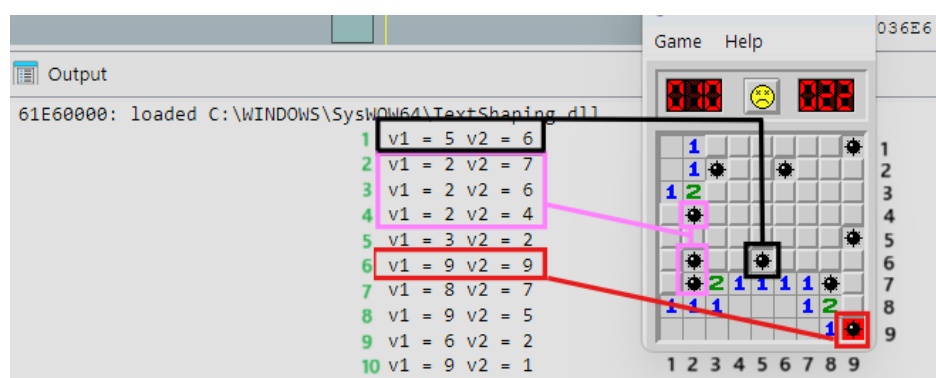


Rysunek 3.14: skrypt IDC użyty do podglądnięcia wartości rejestrów podczas uruchomienia programu

Aby to zweryfikować w grze Saper wybieramy tryb „Custom” i ustawiamy różne wartości, dla przykładu wybierzemy wysokość równą 9 oraz szerokość równą 15. Po takiej zmianie ustawień, jak widać na powyższym obrazku, dowiedliśmy, że zmienna `dword_1005334` odpowiada za szerokość (liczbę kolumn) planszy a zmienna `dword_1005338` za wysokość (liczbę rzędów) planszy. Zmienimy nazwę tych zmiennych na odpowiednio „width” oraz „height” dla dokumentacji. Skoro zmienne te ograniczają szerokość i wysokość planszy to oznacza, że zmienne `v1` i `v2` przyjmują odpowiednio wartości z zakresów  $\langle 1, \text{width} \rangle$  oraz  $\langle 1, \text{height} \rangle$ . Skoro to jedyne miejsce gdzie program losuje jakiegokolwiek liczby a na dodatek losuje je z ograniczeniem zakresów do wysokości oraz szerokości pola to dowiedliśmy, że zmienne `v1` oraz `v2` przechowują indeksy losowych pól na planszy gry. Postawmy hipotezę, że zmienne `v1` oraz `v2` przechowują podczas jednego uderzenia breakpoint indeksy pozycji jednej z min na planszy. Zweryfikujemy tę hipotezę tak jak zbadaliśmy zmienne `dword`, ustawiając breakpoint ze skryptem wyświetlającym przypisywaną wartość.

### 3.3. Manualna rekonstrukcja mapy pola minowego gry Saper

Przywrócono grę na tryb „Beginner” gdzie na planszy rozmieszczone jest 10 min, wysokość i szerokość planszy na tym poziomie trudności to 9. Po ustawieniu odpowiednich breakpointów ze skryptami, analogicznie do poprzednich, tym razem na zmiennych v1 oraz v2 otrzymujemy wedle wcześniejszych znalezisk wartości większe lub równe 1 i mniejsze lub równe 9. Nadal pozostaje pytanie, jak liczone są indeksy kolumn i rzędów? Można je przecież liczyć na różne sposoby. Korzystając, z tego, że jesteśmy w procesie analizy dynamicznej, rozegramy partię gry którą szybko celowo przegramy, bowiem po przegranej grze Saper ujawni nam lokalizację wszystkich min i będziemy wtedy w stanie zobaczyć jak indeksowane są rzędy i kolumny. Po przegranej rozgrywce mapujemy wylosowane indeksy do odpowiednich min na planszy. Po krótkim, manualnym, przypisywaniu indeksów pozycji min do planszy uzyskujemy odpowiedź na nasze pytanie. Potwierdziliśmy, że indeksy rzędów liczone są, rozpoczynając od 1, od góry, rosnąco do dołu, a indeksy kolumn liczone są, rozpoczynając od 1, od lewej rosnąco do prawej.



Rysunek 3.15: mapowanie przechwyconych wartości rejestrów (indeksów planszy) do pola minowego gry Saper

Ponadto, zmienne v1 oraz v2 zostały przypisane 10 razy podczas jednej rozgrywki – czyli dokładnie tyle razy ile jest min w trybie „Beginner”. Dla dokumentacji zmieniono więc nazwy v1 oraz v2 na odpowiednio „mineColumnIdx” oraz „mineRowIdx”.

Po udowodnieniu, że zmienne mineColumnIdx oraz mineRowIdx definiują lokalizację min, analiza pseudokodu w IDA ukazuje kluczową operację programu.

W linii 27 pseudokodu (rys. 3.16) następuje instrukcja: „byte\_1005340[32 \* mineRowIdx + mineColumnIdx] |= 0x80u;”. Wyrażenie 32 \* rząd + kolumna to sposób na odwzorowanie tablicy dwuwymiarowej w jednowymiarowym bloku pamięci RAM. Szerokość wier-

```

19  do
20  {
21      do
22      {
23          mineColumnIdx = rand_0(width) + 1;
24          mineRowIdx = rand_0(height) + 1;
25      }
26      while ( byte_1005340[32 * mineRowIdx + mineColumnIdx] < 0 );
27      byte_1005340[32 * mineRowIdx + mineColumnIdx] |= 0x80u;
28      --dword_1005330;
29  }

```

Rysunek 3.16: pseudokod C, oznaczona instrukcja linii 27

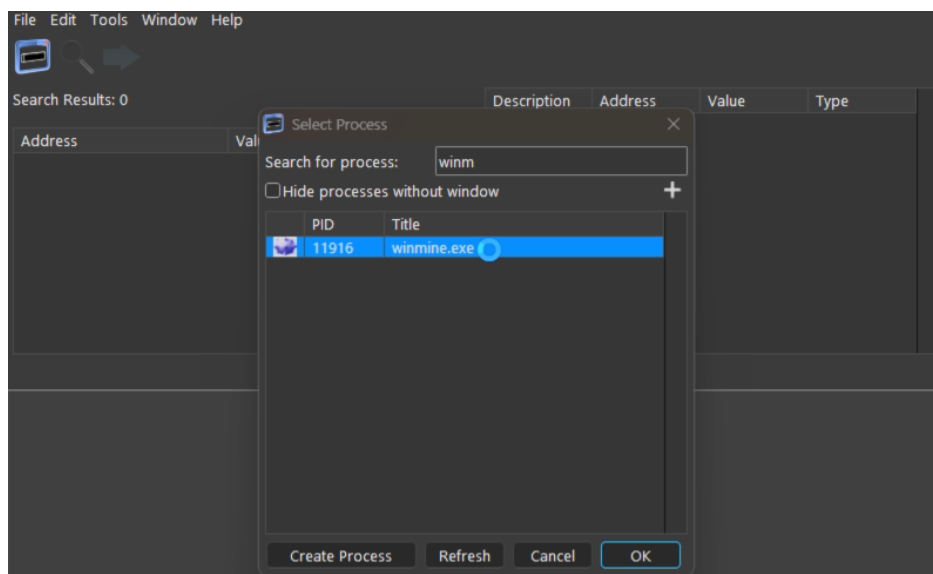
sza w pamięci zawsze wynosi 32 bajty (niezależnie od poziomu trudności w grze). Linie 26-27 to ostatnie linie pseudokodu funkcji odnoszące się do zmiennych `mineRowIdx` i `mineColumnIdx`. W obu przypadkach są one użyte do indeksowania tablicy `byte_1005340`. Linia 26 to warunek wykonywania pętli losującej indeksy, a linia 27 dokonuje modyfikacji wartości tablicy `byte_1005340` na konkretnym indeksie wyliczonym za pomocą wylosowanych wcześniej wartości. Z dużą pewnością możemy powiedzieć, że `byte_1005340` to interesująca nas tablica przechowująca informacje o każdej komórce pola minowego. Ponadto, skoro to tablica bajtów to możemy postawić tezę, że każdej komórce został poświęcony jeden bajt. Spostrzegliśmy również w jaki sposób oznaczane są komórki z minami w pamięci gry. Wyrażenie „`|= 0x80u`” czyli przypisanie wyniku operacji bitowej OR na wartości hexadecymalnej `0x80` w rezultacie ustawia najwyższy bit bajtu, co oznacza, że wartość `0x80` (lub wyższa) pod danym adresem sygnalizuje obecność miny w komórce. Pamięć operacyjna, do której wpisywane są miny, jest reprezentowana w IDA przez statyczny adres bazy danych `byte_1005340`. Mając pewność co do tego adresu, możemy przejść z analizy dezasemblera bezpośrednio do skanera pamięci.

W celu weryfikacji struktury planszy w czasie rzeczywistym użyto oprogramowania CrySearch. Ponieważ system Windows XP (z którego pochodzi badana aplikacja) domyślnie łąduje pliki wykonywalne `.exe` pod adres bazowy `0x01000000`, adres widoczny w IDA jako `0x01005340` przekłada się na relatywne przesunięcie (z ang. offset) rzędu `0x5340` względem początku modułu pamięci gry w procesie.

Po potwierdzeniu struktury planszy, kolejnym krokiem analitycznym było zdekodowanie znaczenia poszczególnych bajtów reprezentujących komórki gry. W tym celu zastosowano korelację stanu interfejsu graficznego ze stanem pamięci operacyjnej w czasie rzeczywistym.

Na samym początku, uruchomiono program CrySearch i za pomocą opcji `File -> Open Process` połączono skaner pamięci do uruchomionej instancji gry Saper – procesu „`winmine.exe`”

Następnie korzystając z wiedzy zdobytej w poprzednim podrozdziale, obliczono relatyw-



Rysunek 3.17: otwarcie procesu „winmine.exe” za pomocą programu CrySearch

ne przesunięcie (ang. offset) adresu pamięci pierwszej komórki planszy od bazowego adresu planszy, podstawiając za obie zmienne liczbę 1.

Offset komórki pierwszego rzędu i pierwszej kolumny:

$$32 \cdot (\text{mineRowIndex} = 1) + (\text{mineColumnIdx} = 1) = 33 = 0x21$$

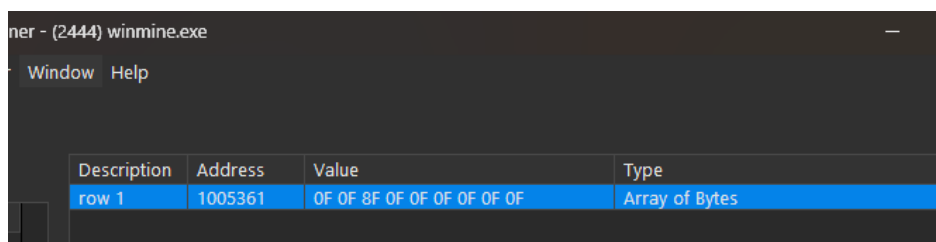
Adres tablicy pola minowego zaczyna się na 0x1005340. Z arytmetyki wskaźników wynika, że dodając do tego adresu wyliczone przesunięcie (adresacja tablicy) uzyskamy adres pierwszej komórki planszy. Na podstawie powyższego, sformułowano wzór pozwalający na obliczenie adresu w pamięci dowolnej komórki planszy. Wzór na adres dowolnej komórki planszy o indeksach (rzęd, kolumna):

$$\text{Adres} = 0x1005340 + (32 \cdot \text{rzęd} + \text{kolumna})$$

Dla komórki o indeksach 1, 1 wyliczono ze wzoru adres 0x1005361. Wyliczony adres dodano do programu CrySearch za pomocą opcji Manually add address i wybrano typ danych „Array of bytes” o długości 9 (ilość komórek w rzędzie na poziomie trudności „Beginner”).

Jak widoczne powyżej, dla tej konkretnej instancji gry Saper, trzecia komórka zawiera minę. Możemy wysnuć taką informację ponieważ wartość bajta odpowiadającego za trzecią komórkę planszy jest większa lub równa 0x80, co jest tożsame z obecnością miny, jak pokazano w podrozdziale 3.1. Następnie poddano ciągłej obserwacji wyodrębniony bufor planszy (pierwszy rząd). W oknie gry przeprowadzono serię kontrolowanych interakcji, takich jak:

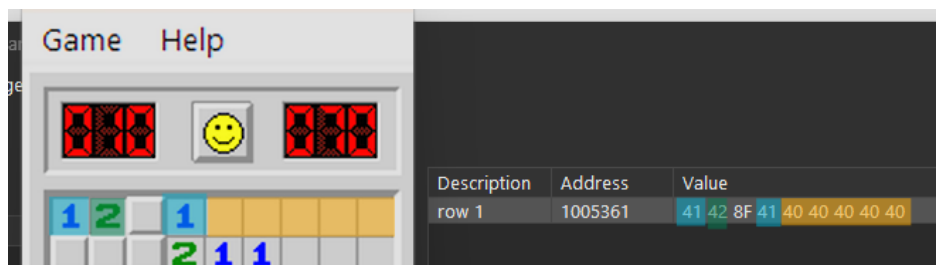
- postawienie flagi na nieodkrytym polu



Rysunek 3.18: dodanie adresu pola minowego do tabeli programu CrySearch

- odsłonięcie pola pustego
- odsłonięcie pola sąsiadującego z minami (zawierającego cyfrę)

Rejestrując i porównując wartości pojedynczych bajtów przed wykonaniem akcji oraz natychmiast po niej, zidentyfikowano jednoznaczne mapowanie stanów wizualnych gry na ich reprezentację liczbową w pamięci RAM. Zaobserwowano na przykład, że komórka domyślnie przyjmuje wartość 0x0F (pole nieodkryte), natomiast postawienie flagi przez gracza skutkuje natychmiastową nadpisaniem tego bajtu wartością 0x0E. Zaobserwowano również, że komórka zawierająca minę przyjmuje wartość 0x8F. Z kolei pola odkryte przyjmują wartości odpowiadające liczbie sąsiadujących min (np. 0x41 dla jedynki, 0x42 dla dwójki).



Rysunek 3.19: obserwacja bajtów pierwszego rzędu pola minowego w pamięci

Taka metoda analizy (obserwacja zmian w pamięci wywoływanych akcjami gracza) pozwoliła na stworzenie tabeli wszystkich możliwych stanów komórki planszy gry. Zrozumienie tych wartości jest warunkiem koniecznym do późniejszej, celowej manipulacji pamięcią opisywanej w rozdziale 4.

Zidentyfikowano dwie kluczowe flagi bitowe, określające właściwości danego pola w logice gry:

Dolne 5 bitów służy natomiast do opisu stanu wizualnego komórki. Zestawienie tych stanów bazowych prezentuje się następująco:

Wartości z powyższych tabel nie występują w pamięci operacyjnej w izolacji. Ostateczny bajt komórki jest wynikiem nałożenia na siebie (poprzez logiczną operację OR) właściwości fizycznej oraz stanu wizualnego.



Tabela 3.1: Kluczowe flagi bitowe określające właściwości pola w logice gry Saper

| Wartość | Znaczenie w logice gry            |
|---------|-----------------------------------|
| 0x40    | Pole zostało odkryte przez gracza |
| 0x80    | Pole, pod którym ukryta jest mina |

Oznacza to, że odczytywane przez skaner pamięci wartości są precyzyjnymi sumami logicznymi:

Przykładowo:

Odkryte puste pole: Stan pusty (0x00) połączony z flagą odkrycia (0x40) daje w pamięci wartość 0x40.

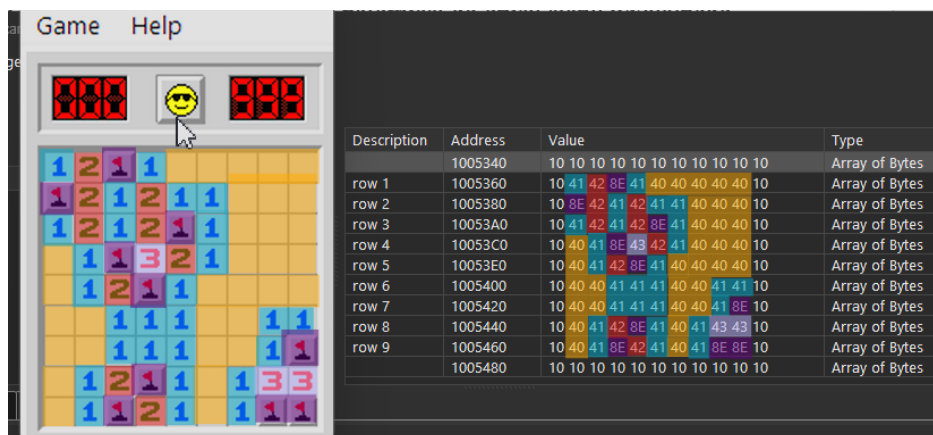
Oraz dalej analogicznie:

Odkryta cyfra 1: (0x01) OR (0x40) = 0x41

Nieodkryta mina: (0x0F) OR (0x80) = 0x8F

Oznaczona flagą mina: (0x0E) OR (0x80) = 0x8E

Poniższy zrzut ekranu potwierdza tę teorię:



Rysunek 3.20: pełne zmapowanie bajtów pola minowego do pamięci procesu gry

Na ten moment zdobyto już wystarczająco informacji, żeby zaimplementować funkcjonalność podglądu mapy pola minowego do aplikacji oszukującej do gry Saper. Taka mapa pozwoli wygrać każdą partię Sapera w znacznie krótszym czasie niż miałyby to miejsce bez niej (opis implementacji został zawarty w podrozdziale 4.2). W następnym podrozdziale, idąc o krok dalej, zdobyto wiedzę potrzebną do implementacji funkcjonalności automatycznego umieszczenia flag na każdej z tych komórek, które zawierają miny.

Tabela 3.2: Zestawienie wszystkich możliwych stanów bajtu komórki pola minowego gry Saper

| Wartość     | Znaczenie w logice gry                         | Dodatkowe wyjaśnienia                                                                                                                                                                            |
|-------------|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0x00        | Pole puste (bezpieczne)                        | brak min w sąsiedztwie                                                                                                                                                                           |
| 0x01 - 0x08 | Pole z cyfrą (bezpieczne)                      | cyfra od 1 do 8; cyfra wskazuje ilość min sąsiadujących z polem                                                                                                                                  |
| 0x0A        | Pole z miną (przegrana)                        | mina nieodkryta ani nie oznaczona flagą przez gracza; ujawniana na koniec przegranej gry                                                                                                         |
| 0x0B        | Pole niepoprawnie oznaczone flagą (bezpieczne) | błędnie oznaczone flagą, jako zawierające minę                                                                                                                                                   |
| 0x0C        | Pole z miną (przegrana)                        | błędne odkrycie niebezpiecznego pole zawierającego minę; skutkujące detonacją (przegraną); występuje tylko na polu, którego odkrycie zakończyło grę                                              |
| 0x0D        | Pole ze znakiem zapytania                      | oznaczone przez gracza jako niepewne za pomocą prawego przycisku myszy;                                                                                                                          |
| 0x0E        | Pole z flagą                                   | oznaczone przez gracza jako zawierające minę za pomocą prawego przycisku myszy; poprawne oznaczenie flagą każdego z uzbrojonych pól (oraz odkrycie wszystkich bezpiecznych pól) skutkuje wygraną |
| 0x0F        | Pole nieodkryte                                | stan domyślny                                                                                                                                                                                    |
| 0x10        | Margines planszy gry                           | niewidoczna ramka ograniczająca planszę                                                                                                                                                          |

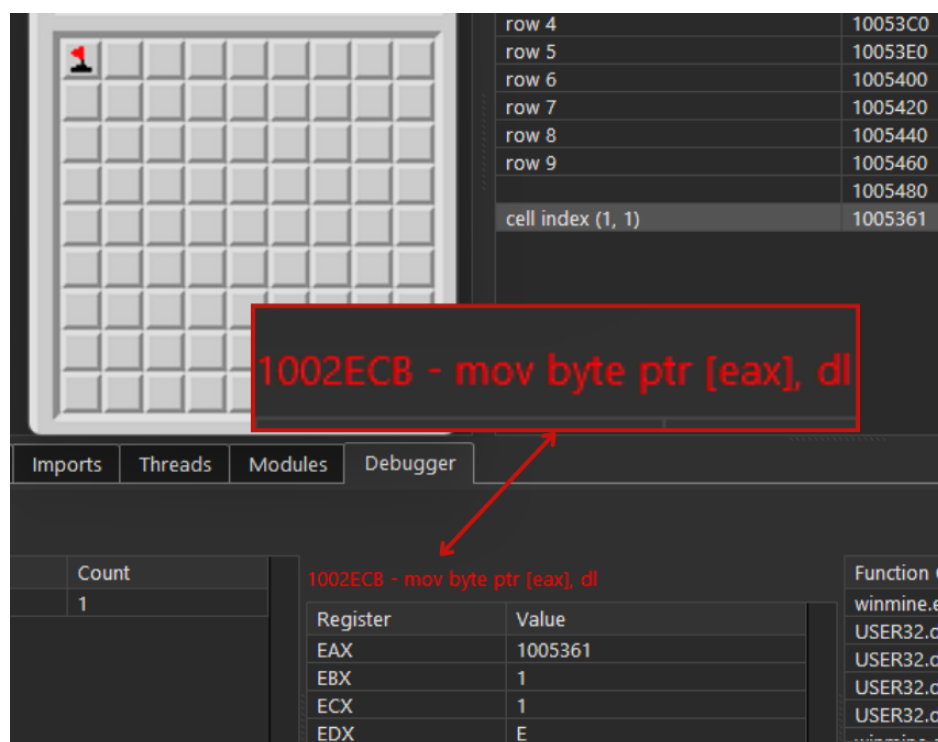
### 3.4. Dekompilacja funkcji odpowiedzialnych za zmianę stanu pola

W poprzednim podrozdziale obszernie odpowiadano na pytania dotyczące ułożenia pola minowego w pamięci gry. Skupiając się na drugiej, planowanej funkcjonalności automatycznego oznaczania min, zbadano wewnętrzną funkcję gry Saper umieszczającą flagę na danej komórce, w celu późniejszego wywołania jej za pomocą oprogramowania oszukującego. Kiedy omawiana funkcja zostanie wywołana dla każdej z min na planszy to wysiłek potrzebny ze strony gracza, aby wygrać daną partię Sopera zostanie zredukowany jedynie do odkrycia bezpiecznych pól.

W tym celu powrócono do skanera pamięci CrySearch, na którym zakończono analizę w poprzednim podrozdziale. Użyto również stworzonego uprzednio pliku `winmine_crysearch.csat`

W podrozdziale 3.2, podczas badania funkcji „`rand_0`”, chcieliśmy ustalić jakie wartości są przekazywane jako argumenty podczas jej wywołań. W tym celu, za pomocą środowiska IDA, dołączyliśmy do procesu gry Saper debugger i postawiliśmy breakpointy na konkretnych instrukcjach kodu maszynowego. Udało nam się ustalić wtedy, że są to zmienne wyznaczające wymiary planszy gry. Podobnie jak w tamtym przypadku, ale od nieco innej strony, chcemy ustalić teraz adres instrukcji nadpisującej wartości komórek pola minowego. W tym celu, postawimy breakpoint na adresie w pamięci jednej z komórek. Wybrano komórkę o indeksach (1, 1), czyli komórkę widoczną w lewym górnym rogu planszy w interfejsie graficznym gry. Ponieważ w poprzednim podrozdziale zrekonstruowaliśmy mapę pola minowego z pamięci gry, mamy w programie CrySearch poukładane wierszami wycinki 11 bajtów, czyli każdy rząd planszy (początkowej planszy  $9 \times 9$ ). „Pole minowe” zaczyna się w pamięci na adresie `0x1005340` dla której wyliczono adres w pamięci `0x1005361` za pomocą programu CrySearch, podłączono debugger do procesu gry o nazwie „`winmine.exe`”. Następnie w widoku podglądu i edycji zmiennych w pamięci dodano interesujący nas adres pojedynczej, dowolnej komórki, ustawiono breakpoint typu „Write” na.. Wybrano opcję `Set Breakpoint -> Write` i powrócono do interfejsu graficznego gry a następnie kliknięto prawym przyciskiem myszy na pole w lewym górnym rogu planszy (komórka o indeksach (1, 1)).

Jak widać na rysunku powyżej, po oznaczeniu flagą komórki o indeksach (1, 1), breakpoint na adresie przechowującym bajt, reprezentujący stan komórki, został aktywowany przez instrukcję o adresie `1002ECB`. Dla czytelności i ułatwienia poprzez funkcjonalność dekompiletora, powrócono do pliku projektowego środowiska IDA. Na widoku dezasemblacji, przemieszczono się do interesującego nas adresu za pomocą opcji wywołanej przy użyciu skrótu klawiszowego „G”. W oknie „Jump to address” wybrano adres instrukcji `1002ECB`.



Rysunek 3.21: CrySearch, wywołany breakpoint zapisu interesującej nas zmiennej

Po przejściu do widoku dezasemblacji, na interesującym nas adresie możemy zaobserwować, że instrukcja 0x1002ECB, która aktywowała breakpoint, jest częścią funkcji rozpoczynającej się na adresie 0x1002EAB. Funkcja ta (sub\_1002EAB) wydaje się więc idealnym kandydatem na zdalne wywołanie w celu zmiany stanu logicznego oraz graficznego konkretnej komórki. Podczas zdobywania informacji potrzebnych do zdalnego wywołania badamy sygnaturę funkcji i...natrafiamy na istotny problem. Mianowicie, procedura sub\_1002EAB przyjmuje 3 argumenty. Dwa z nich są typu liczby całkowitej, co biorąc pod uwagę kontekst (funkcja wywoływana jest dla konkretnej komórki), najprawdopodobniej wskazuje na to, że argumenty typu int, tak jak poprzednio, wyznaczają indeksy rzędu i kolumny komórki, dla której wykonywana jest funkcja. Jednak ostatni z nich jest typu „char”, używanego zwykle do przechowywania pojedynczego znaku alfabetu. Typ danych char jest wielkości jednego bajta, a więc równy wielkości pojedynczego elementu tablicy pola minowego. Możemy zatem przypuszczać, że wyznacza on nowy stan pola jaki będzie definiował komórkę po wykonaniu funkcji lub maskę bitową do połączenia z aktualnym stanem pola w celu uzyskania stanu po modyfikacji. Aby rozwiać wątpliwości, możemy, tak jak poprzednio, przechwycić wartość zmiennej podczas interakcji z planszą. Zanim jednak to zrobimy, spójrzmy najpierw na wszystkie odniesienia do adresu 0x1002EAB w programie. W tym celu, korzystamy ponownie z widoku referencji krzyżowych (xrefs) dostępnego pod skrótem klawiszowym „X”.

```

sub_1002EAB
sub_1002EAB
sub_1002EAB
sub_1002EAB      ; int __stdcall sub_1002EAB(int, int, char)
sub_1002EAB      sub_1002EAB proc near
sub_1002EAB
sub_1002EAB      arg_0= dword ptr 4
sub_1002EAB      arg_4= dword ptr 8
sub_1002EAB      arg_8= byte ptr 0Ch
sub_1002EAB
sub_1002EAB      mov     eax, [esp+arg_4]
sub_1002EAB+4    mov     ecx, [esp+arg_0]
sub_1002EAB+8    push    [esp+arg_4]
sub_1002EAB+C    shl     eax, 5
sub_1002EAB+F    lea     eax, b_minefield[eax+ecx]
sub_1002EAB+16   mov     dl, [eax]
sub_1002EAB+18   and     dl, 0E0h
sub_1002EAB+1B   or      dl, [esp+4+arg_8]
sub_1002EAB+1F   push    ecx
sub_1002EAB+20   mov     [eax], dl
sub_1002EAB+22   call    sub_1002646
sub_1002EAB+27   retn    0Ch
sub_1002EAB+27   sub_1002EAB endp
sub_1002EAB+27

```

Rysunek 3.22: widok dezasemblacji funkcji sub\_1002EAB

Okno xrefs wskazuje na trzy różne miejsca w kodzie maszynowym, gdzie wywoływany jest analizowany podprogram. Konkretniej, sub\_1002EAB jest wywoływany przez instrukcje znajdujące się wewnątrz procedur o adresach:

1. 0x1003512
2. 0x10035B7
3. 0x100374F

Dla czytelności, w poniższym tekście będziemy odnosić się do tych procedur używając przypisanych powyżej cyfr lub offsetu względem adresu bazowego pliku. (ImageBase – PE file)

Na powyższym rysunku możemy zaobserwować ciekawą zależność. Funkcje 1. (offset 0x3512) oraz 2. (offset 0x35B7) wywołują sub\_1002EAB, zawsze przekazując przy tym sztywno ustaloną wartość 0x4C jako trzeci argument.

Po krótkim wglądzie do tabeli wszystkich możliwych stanów komórki planszy (powstałej w ramach podrozdziału 3.3), możemy zaobserwować podobieństwo wartości 0x4C do dwóch flag bitowych: 0x40 (pole zostało odkryte przez gracza) oraz 0x0C (pole z miną, wartość 0x0C jest widoczna w pamięci tylko w przypadku przegranej przez odkrycie uzbrojonego pola).

| 1. | 0x1003512                      |
|----|--------------------------------|
| 16 | else if ( dword_10057A4 )      |
| 17 | {                              |
| 18 | sub_1002EAB(a1, a2, 0x4C);     |
| 19 | return sub_100347C(0);         |
| 20 | }                              |
| 2. | 0x10035B7                      |
| 32 | else                           |
| 33 | {                              |
| 34 | v8 = 1;                        |
| 35 | sub_1002EAB(i, v3, 0x4C);      |
| 36 | }                              |
| 37 | }                              |
| 3. | 0x100374F                      |
| 26 | }                              |
| 27 | sub_1002EAB(a1, a2, v4);       |
| 28 | if ( (*v2 & 0x1F) == 0xE && dv |
| 29 | sub_100347C(1);                |
| 30 | }                              |

Rysunek 3.23: widok xrefs funkcji sub\_1002EAB

Dodatkowo, tak jak w przypadku opisanych wcześniej przykładów, ostateczny bajt komórki jest wynikiem nałożenia na siebie (poprzez logiczną operację OR) właściwości fizycznej oraz stanu wizualnego. Możemy więc założyć, że wartość 0x4C występującą w funkcjach 1. i 2. powstała na skutek operacji przedstawionej poniżej. Odkryta mina (detonacja; przegrana):

$$(0x0C) \text{ OR } (0x40) = 0x4C$$

Z kolei funkcja 3. (offset 0x374F), nie używa żadnej ustalonej na sztywno wartości jako parametr przekazywany do sub\_1002EAB. Zamiast tego, wykonuje różne instrukcje warunkowe i uzależnia od nich ostatecznie przekazany parametr.

Powierzchnie analizując pseudokod funkcji 3. o offsecie 0x374F, możemy potwierdzić dwie z możliwych ostatecznych wartości argumentu przekazywanego do sub\_1002EAB. Są to wartości 0x0E oraz 0x0F. To, która z wartości zostanie przekazana jako argument, jest determinowane przez wykonaną ścieżkę instrukcji warunkowych. Wartości te, również są nam znane z poprzedniego podrozdziału. 0x0E

W ramach ostatecznej weryfikacji, że to funkcja sub\_100374F jest odpowiedzialna za oznaczenie pola flagą lub znakiem zapytania, do gry podłączono debugger środowiska IDA

```

13     if ( v3 == 0xE )
14     {
15         v4 = 2 * (*(__DWORD *)&dword_10056BC == 0) + 13;
16         sub_100346A(1);
17     }
18     else if ( v3 == 0xD )
19     {
20         v4 = 0xF;
21     }
22     else
23     {
24         v4 = 0xE;
25         sub_100346A(-1);
26     }
27     sub_1002EAB(a1, a2, v4);

```

Rysunek 3.24: pseudokod C funkcji o adresie 0x100374F

oraz ustawiono breakpointy na adresach każdej z funkcji: 1. (offset 0x3512), 2. (offset 0x35B7), 3. (offset 0x374F). Następnie kliknięto prawym przyciskiem myszki na dowolną komórkę i zaobserwowano wywołanie tylko jednego z breakpointów – breakpointu na adresie należącym do funkcji sub\_100374F. W ramach dokumentacji nadano jej zatem, w pliku projektowym IDA, bardziej przyjazną nazwę „onRightClickField(int row, int column)”.

## 3.5. Podsumowanie

Powyższy rozdział stanowi dokumentację procesu analizy statycznej i dynamicznej pliku wykonywalnego gry. W celu utrwalenia wiedzy, w poniższym tekście podsumowano najważniejsze osiągnięcia pochodzące z powyższych analiz.

Proces analityczny został ukierunkowany na zebranie informacji potrzebnych do stworzenia programu oszukującego do gry Saper w rozdziale 4. W podrozdziale 2.3 wyznaczono pierwszy, początkowy cel analizy jako „znalezienie miejsca w pamięci gdzie przechowywane są informacje o planszy (polu minowym)”.

Wychodząc z punktu startowego, na samym początku przeprowadzono eksperyment myślowy na temat implementacji pola minowego z punktu widzenia inżynierii oprogramowania. Wynikało z niego, że twórcy gry musieli skorzystać z generatora liczb pseudolosowych. Podczas wstępnej analizy statycznej, w tabeli adresów importu (IAT) zidentyfikowano w kodzie maszynowym gry Saper odniesienia do funkcji `rand()` oraz `srand()` pochodzących z biblioteki `msvcrt.dll` (Microsoft C Runtime Library). Następnie sprawdzono, że jedyne wywołanie importowanej funkcji `rand()` występuje w funkcji o adresie `0x1003940`. Dalsza analiza tej funkcji doprowadziła nas do stworzenia jej pełnej dokumentacji i przemianowania na `rand_0`. Analizując wywołania, udokumentowanej przez nas, funkcji `rand_0` dotarliśmy do jedyne miejsca w kodzie maszynowym gry, gdzie generowane są liczby pseudolosowe, czyli do funkcji o adresie `0x100367A`. Postawiono hipotezę, że to właśnie w obrębie tej funkcji losowane są pozycje min na planszy.

Kontynuując, w ramach zbadania postawionej hipotezy, przeprowadzono analizę dynamiczną za pomocą debuggera podłączonego do instancji gry Saper. Z użyciem skryptów z natywnego dla środowiska IDA języka IDC, podczas wykonania programu, podglądnięto wartości zmiennych użytych do ograniczenia zakresu wygenerowanej liczby pseudolosowej czyli zmiennych `dword_1005334` oraz `dword_1005338`. Potwierdzono, że przechowują one odpowiednio szerokość oraz wysokość planszy gry.

Następnie skupiono się na rekonstrukcji mapy pola minowego gry Saper. Pod lupę wzięto zmienne `v1` oraz `v2`, do których przypisywane są wspomniane wyżej, generowane liczby pseudolosowe, ograniczone przez odpowiednio szerokość oraz wysokość pola minowego. Po ponownym użyciu skryptów, tym razem dla zmiennych `v1` i `v2`, potwierdzono, że odpowiadają one za indeksy kolumny i rzędu wylosowanej pozycji miny. Dla dokumentacji, zmienne `v1` oraz `v2` przemianowano, odpowiednio, na `mineColumnIdx` oraz `mineRowIdx`. Pozostając we wnętrzu funkcji o adresie `0x100367A` zidentyfikowano kluczową strukturę pola minowego oraz jej adres. Ustalono, że gra przechowuje pole minowe w pamięci, za pomocą tablicy bajtów dostępnej pod adresem `0x1005340`. Wyprowadzono również wzór na adres dowolnej



komórki planszy w postaci  $0x1005340 + (32 \cdot \text{rz\ddot{a}d} + \text{kolumna})$ . Następnie, z użyciem skanera pamięci CrySearch, zmapowano wszystkie możliwe stany komórki (wartości bajtów) i opisano w tabeli 3.3. Na tym etapie analizy zdobyto wszelkie, niezbędne informacje potrzebne do implementacji w programie oszukującym funkcjonalności podglądu planszy gry.

Uwagę skierowano więc na zebranie informacji potrzebnej do implementacji funkcjonalności automatycznego oznaczania flagą pól zawierających minę. W celu zminimalizowania nakładów pracy oraz zapewnienia braku ingerencji programu oszukującego w stabilność gry, zdecydowano się skorzystać z jednej z istniejących w grze funkcji, która będzie przez program oszukujący zdalnie wywoływana w ramach zrealizowania funkcjonalności. Po przeprowadzeniu odpowiednich badań (szczegółowo opisanych w podrozdziale 3.4), ustalono, że za oznaczanie pola flagą po naciśnięciu prawego przycisku myszy, odpowiedzialna jest funkcja o offsecie `0x374F`. Funkcję udokumentowano i przemianowano na „`onRightClickField(int row, int column)`”.

W następnym rozdziale udokumentowano proces rozwoju autorskiego oprogramowania oszukującego do gry Saper. Udokumentowane w tym rozdziale znaleziska znacząco ułatwiają proces tworzenia takiego oprogramowania. Bez zebrania informacji w ramach powyższych analiz, stworzenie takiego oprogramowania byłoby znacznie trudniejsze, o ile nie niemożliwe.